

Milestone 1

DONE

EML Viewer Chrome Extension · Scaffold + File Ingestion + Parsing

In plain English

We built the **skeleton of the extension** and got the single hardest "does this even work" question answered: *can we take a `.eml` email file, open it, and pull out the real contents?* The answer is yes.

Right now the extension gives you a page where you **drop an email file**, and it shows you what's inside it — who it's from, the subject, the date, whether it has an HTML body, and how many attachments it has. It doesn't *display* the email nicely yet (that's the next milestone). Think of this milestone as **"the plumbing is connected and water comes out."**

What an "extension" even is (quick grounding)

A Chrome extension is just a bundle of web files (HTML, CSS, JavaScript) plus a `manifest.json` — a settings file that tells Chrome "here's my name, my icon, what I'm allowed to do, and what happens when someone clicks my toolbar button." Our manifest says: when clicked, open our import page; the only permission we ask for is `storage` (to remember emails locally); and we explicitly ask for **no internet access at all**.

What we actually built this milestone

Piece	What it does
<code>manifest.json</code>	The extension's ID card and permission list (storage only, no network).
<code>service-worker.js</code>	The tiny always-available script that opens the import page when you click the toolbar icon.
import page	The drag-and-drop / file-picker screen where you load an <code>.eml</code> file. Currently shows a debug panel of what was parsed.
<code>parse-eml.js</code>	The wrapper around <code>postal-mime</code> , the library that does the heavy lifting of decoding the raw email format.
viewer / attachment pages	Placeholder shells for now — filled in during Milestones 2 and 4.

How it works (a bit more detail)

When you drop a file, the import page reads it as raw bytes (`file.arrayBuffer()`), hands those bytes to `postal-mime`, and gets back a structured object: `from`, `to`, `subject`, `date`, an HTML body, a plain-text body, and a list of attachments (each with its filename, type, and size). We then show a readable summary plus the raw JSON so we can confirm parsing is correct before we trust it.

Why a separate "parse" step at all? Email files are surprisingly messy — multiple body formats, weird encodings, nested attachments. `postal-mime` is a well-tested library that handles that so we don't have to write fragile parsing ourselves.

Why a build tool (Vite)?

The parsing library is shipped as modern JavaScript modules. Chrome extensions need everything packaged a specific way, so Vite + the CRXJS plugin bundles our code and rewrites file paths into a clean `dist/` folder. We load *that* folder into Chrome, not our raw source.

Proof it works

- **Automated test passes:** a unit test feeds a sample multipart email through the parser and confirms it extracts the subject, sender, and both body formats. (`npm test` → 2 passed.)
- **Production build succeeds:** `npm run build` produces a complete `dist/` with the manifest, all three pages, and the parser bundle.

What is NOT done yet (on purpose)

- No nice rendering of the email body — that's Milestone 2, along with the security sandbox.
- No saving emails / no opening them in a dedicated viewer tab yet.
- No inline images or attachment previews — Milestones 3 and 4.
- Icons are plain blue placeholder squares for now.

✓ Your 2-minute check in Chrome

1. Open Chrome → `chrome://extensions` → turn on **Developer mode** (top-right).
2. Click **Load unpacked** and select the `dist/` folder in the project.
3. Click the extension's icon in the toolbar — the **Import page** should open in a new tab.

4. Drag the file `samples/basic.eml` onto the drop zone (or click to pick it).
5. You should see: subject "Welcome to the EML Viewer test", the From/To/Cc, "Has HTML body: true", "Attachments: 0", and a JSON panel.

If that all shows up, Milestone 1 is confirmed and we move to Milestone 2 (secure rendering in a sandboxed viewer tab).

One thing we fixed behind the scenes

Your machine is behind a corporate network that inspects secure traffic, which initially blocked `npm` from downloading anything (a certificate-trust error). We pointed `npm` at the macOS system certificate store, which contains your company's trusted certificate, and saved that setting so it won't recur. We also had to approve one install script that Node 26 blocks by default. Nothing to do on your end — just noting it.